

The ebproofx package ebproof extended

Jay Lee*

Version 1.0.0 – Released 2024-01-10

Contents

	A	Implementation	3
1	Introduction	1	3
	A.1	Parameters	3
	A.2	Proof boxes	5
2	Environments	2	9
	A.3	Making inferences	9
	A.4	Stack-based interface	10
3	Statements	2	13
	A.5	Document interface	13

1 Introduction

The ebproofx package provides an ergonomic and scalable interface for typesetting proof trees. However, it has been a long-standing issue to stack hypotheses using ebproof.

This is an extended work of [ebproof](#) that allows users to stack multiple hypotheses vertically¹. It generalizes `\infer` to accept a comma-separated list of integers, and a single integer works as well. Moreover, I believe this will reduce, if not resolve, a need to make ebproof compatible with the [mathpartir](#) environment².

In addition, it provides a wrapper environment `InfRule` that typesets the rule names more space-efficiently. Here is an example³:

$\frac{\begin{array}{c} \text{TA-APP} \\ \Gamma \vdash t_1 : T_1 \quad \Gamma \vdash T_1 \uparrow (T_{11} \rightarrow T_{12}) \\ \Gamma \vdash t_2 : T_2 \quad \Gamma \vdash T_2 <: T_{11} \end{array}}{\Gamma \vdash t_1 t_2 : T_{12}}$	<pre>\begin{InfRule}{TA-App} \hypo{\Gamma \vdash t_1 : T_1} \hypo{% \Gamma \vdash t_2 : T_2 \Gamma \vdash T_1 \uparrow (T_{11} \rightarrow T_{12}) \Gamma \vdash T_2 <: T_{11} } \infer{2, 2}{\Gamma \vdash t_1 t_2 : T_{12}} \end{InfRule}</pre>
--	--

The web page of the project is at <https://github.com/Zeta611/ebproofx>.

*E-mail: jay.lee@berkeley.edu

¹<https://framagit.org/manu/ebproof/issues/7>, <https://tex.stackexchange.com/q/530939>

²<https://framagit.org/manu/ebproof/issues/6>, <https://github.com/jonsterling/latex-ebproof-rules>

³This example is taken from *Types and Programming Languages* by Benjamin C. Pierce.

2 Environments

Here we only explain the newly introduced environment `InfRule`. Consult the documentation of `ebproof` for other environments.

```

\begin{InfRule}{\langle name \rangle}[\langle alignment \rangle]
  \langle statements \rangle
\end{InfRule}

```

This typesets the rule described by the `\langle statements \rangle`. The `\langle alignment \rangle` provides an alignment specifier for the rule name `\langle name \rangle`—`l` for left, `r` for right, and `c` for center are available. `l` is the default alignment specifier.

$$\frac{\Gamma \vdash t_1 : T_{11} \rightarrow T_{12} \quad \Gamma \vdash t_2 : T_2}{\Gamma \vdash t_1 t_2 : T_{12}} \quad \text{T-APP}$$

```

\begin{InfRule}{T-App}[r]
  \hypo{\Gamma \vdash t_1 : T_{11}} \to T_{12}}
  \hypo{\Gamma \vdash t_2 : T_2}
  \infer2{\Gamma \vdash t_1 t_2 : T_{12}}
\end{InfRule}

```

3 Statements

Here we only explain the extended statement `\infer`. Consult the documentation of `ebproof` for other statements.

```

\infer[\langle options \rangle][\langle arities \rangle][\langle label \rangle]{\langle text \rangle}

```

The statement `\infer` builds an inference step by taking some proof trees from the top of the stack, assembling them with a rule joining their conclusions and putting a new conclusion below. The `\langle arities \rangle` are the comma-separated list of sub-proofs, it may be a list of any number including 0 (in this case there will be a line above the conclusion but no sub-proof). A single number is also accepted, as the original `\infer` command from the `ebproof` package. If `\langle label \rangle` is present, it is used as the label on the right of the inference line; it is equivalent to using the right `label` option.⁴

$\frac{\Gamma \vdash 2 \quad \frac{\Gamma \vdash 1 \quad \Gamma \vdash 3}{\Gamma \vdash 1234} \quad \Gamma \vdash 4}{\Gamma \vdash 1234} \quad \frac{\Gamma \vdash 5 \quad \Gamma \vdash 6}{\Gamma \vdash 7} \quad \frac{\Gamma \vdash 7 \quad \Gamma \vdash 9}{\Gamma \vdash 56789} \quad \Gamma \vdash 8}{\Gamma \vdash 10} \quad \Gamma \vdash 56789}{\Gamma \vdash 12345678910}$	<pre> \begin{prooftree} \hypo{\Gamma \vdash 1} \hypo{\Gamma \vdash 2} \hypo{\Gamma \vdash 3} \hypo{\Gamma \vdash 4} \infer{1,3}{\Gamma \vdash 1234} \hypo{\Gamma \vdash 5} \hypo{\Gamma \vdash 6} \infer{1,3}{\Gamma \vdash 56789} \hypo{\Gamma \vdash 7} \hypo{\Gamma \vdash 8} \hypo{\Gamma \vdash 9} \infer{2,1,2}{\Gamma \vdash 56789} \hypo{\Gamma \vdash 10} \infer{2,1}{\Gamma \vdash 12345678910} \end{prooftree} </pre>
--	--

⁴This paragraph is largely taken from the original `ebproof` package documentation word-by-word.

A Implementation

```

1 <{*package}
2 \NeedsTeXFormat{LaTeX2e}
3 \RequirePackage{expl3}
4 \RequirePackage{xparse}
5 \RequirePackage{relsize}
6 \RequirePackage{tabularray}
7 \ProvidesExplPackage{ebproofx}{2024/01/10}{1.0.0}{ebproof eXtended}
8 <{@=ebproofx}

```

A.1 Parameters

We first declare all options.

```

9 \bool_new:N \l__ebproofx_updown_bool
10 \keys_define:nn { ebproofx } {
11   center .bool_set:N = \l__ebproofx_center_bool,
12   proof~style .choice: ,
13   proof~style / upwards .code:n = \bool_set_false:N \l__ebproofx_updown_bool,
14   proof~style / downwards .code:n = \bool_set_true:N \l__ebproofx_updown_bool,
15   separation .dim_set:N = \l__ebproofx_separation_dim,
16   rule~margin .dim_set:N = \l__ebproofx_rule_margin_dim,
17   rule~thickness .dim_set:N = \l__ebproofx_rule_thickness_dim,
18   rule~separation .dim_set:N = \l__ebproofx_rule_separation_dim,
19   rule~dash~length .dim_set:N = \l__ebproofx_rule_dash_length_dim,
20   rule~dash~space .dim_set:N = \l__ebproofx_rule_dash_space_dim,
21   rule~code .tl_set:N = \l__ebproofx_rule_code_tl,
22   rule~style .choice:,
23   template .tl_set:N = \l__ebproofx_template_tl,
24   left~template .tl_set:N = \l__ebproofx_left_template_tl,
25   right~template .tl_set:N = \l__ebproofx_right_template_tl,
26   left~label .tl_set:N = \l__ebproofx_left_label_tl,
27   right~label .tl_set:N = \l__ebproofx_right_label_tl,
28   left~label~template .tl_set:N = \l__ebproofx_left_label_template_tl,
29   right~label~template .tl_set:N = \l__ebproofx_right_label_template_tl,
30   label~separation .dim_set:N = \l__ebproofx_label_separation_dim,
31   label~axis .dim_set:N = \l__ebproofx_label_axis_dim,
32 }

```

`\ebproofxnewrulestyle` We then define the document-level macro `\ebproofxnewrulestyle` and use it to define the default styles. This simply consists in defining a meta-key.

```

33 \NewDocumentCommand \ebproofxnewrulestyle { mm } {
34   \keys_define:nn { ebproofx } {
35     rule~style / #1 .meta:nn = { ebproofx } { #2 }
36   }
37 }

```

(End of definition for `\ebproofxnewrulestyle`. This function is documented on page ??.)

The styles `simple`, `no rule` and `double` are defined in a straightforward way.

```

38 \ebproofxnewrulestyle { simple } {
39   rule~code = { \tex_hrule:D height \l__ebproofx_rule_thickness_dim }
40 }
41 \ebproofxnewrulestyle { no~rule } {
42   rule~code =

```

```

43 }
44 \ebproofxnewrulestyle { double } {
45   rule~code = {
46     \tex_hrule:D height \l__ebproofx_rule_thickness_dim
47     \skip_vertical:N \l__ebproofx_rule_separation_dim
48     \tex_hrule:D height \l__ebproofx_rule_thickness_dim
49   }
50 }

```

The dashed style uses leaders and filling for repeating a single dash. We use T_EX primitives that have no L^AT_EX3 counterpart for this.

```

51 \ebproofxnewrulestyle { dashed } {
52   rule~code = {
53     \hbox_to_wd:nn { \tex_hsize:D } {
54       \dim_set:Nn \l_tmpa_dim { \l__ebproofx_rule_dash_space_dim / 2 }
55       \skip_horizontal:n { -\l_tmpa_dim }
56       \tex_cleaders:D \hbox:n {
57         \skip_horizontal:N \l_tmpa_dim
58         \tex_vrule:D
59           height \l__ebproofx_rule_thickness_dim
60           width \l__ebproofx_rule_dash_length_dim
61         \skip_horizontal:N \l_tmpa_dim
62       } \tex_hfill:D
63       \skip_horizontal:n { -\l_tmpa_dim }
64     }
65   }
66 }

```

Now we can define the default values, including the default rule style.

```

67 \keys_set:nn { ebproofx } {
68   center = true,
69   proof~style = upwards,
70   separation = 1.5em,
71   rule~margin = .7ex,
72   rule~thickness = .4pt,
73   rule~separation = 2pt,
74   rule~dash~length = .2em,
75   rule~dash~space = .3em,
76   rule~style = simple,
77   template = $\inserttext$,
78   left~template = $\inserttext\mathrel{ }$,
79   right~template = $\mathrel{ }\inserttext$,
80   left~label = ,
81   right~label = ,
82   left~label~template = \inserttext,
83   right~label~template = \inserttext,
84   label~separation = 0.5em,
85   label~axis = 0.5ex
86 }

```

`\ebproofxnewstyle` Defining a style simply means defining a meta-key.

```

87 \NewDocumentCommand \ebproofxnewstyle { mm } {
88   \keys_define:nn { ebproofx } { #1 .meta:n = { #2 } }
89 }

```

(End of definition for \ebproofxnewstyle. This function is documented on page ??.)

A.2 Proof boxes

TeX does not actually provide data structures, so we have to encode things. We provide an allocator for “registers” holding boxes with attributes. Such a register consists in a box register and a property list for marks, which maps mark names to values as explicit dimensions with units.

__ebproofx_new:N Using only public interfaces forces a convoluted approach to allocation: we use a global counter \g__ebproofx_register_int to number registers, then each allocation creates registers named \S__ebproofx_K_N where S is the scope of the register (local or global, deduced from the argument), K is the kind of component (box or marks) and N is the identifier of the register. The proof box register itself only contains the identifier used for indirection.

```

90 \int_new:N \g__ebproofx_register_int
91 \cs_new:Nn \__ebproofx_box:N {
92   \str_item:nn { #1 } { 2 } __ebproofx_ \tl_use:N #1 _box
93 }
94 \cs_new:Nn \__ebproofx_marks:N {
95   \str_item:nn { #1 } { 2 } __ebproofx_ \tl_use:N #1 _prop
96 }
97 \cs_new:Nn \__ebproofx_new:N {
98   \tl_new:N #1
99   \int_gincr:N \g__ebproofx_register_int
100   \str_if_eq:eeTF { \str_item:nn { #1 } { 2 } } { g }
101     { \tl_gset:Nx #1 { \int_to_arabic:n { \g__ebproofx_register_int } } }
102     { \tl_set:Nx #1 { \int_to_arabic:n { \g__ebproofx_register_int } } }
103   \box_new:c { \__ebproofx_box:N #1 }
104   \prop_new:c { \__ebproofx_marks:N #1 }
105 }

```

(End of definition for __ebproofx_new:N.)

__ebproofx_clear:N The box is cleared by setting it to an empty hbox. Using \box_clear:N instead would not work because trying to push this box on the stack would not actually append any box.

```

106 \cs_new:Nn \__ebproofx_clear:N {
107   \hbox_set:cn { \__ebproofx_box:N #1 } {}
108   \prop_clear:c { \__ebproofx_marks:N #1 }
109   \__ebproofx_set_mark:Nnn #1 { left } { 0pt }
110   \__ebproofx_set_mark:Nnn #1 { right } { 0pt }
111   \__ebproofx_set_mark:Nnn #1 { axis } { 0pt }
112 }

```

(End of definition for __ebproofx_clear:N.)

A.2.1 Mark operations

__ebproofx_set_mark:Nnn Setting the value of a mark uses a temporary register to evaluate the dimension expression because values are stored textually in a property list.

```

113 \dim_new:N \l__ebproofx_transit_dim
114 \cs_new:Nn \__ebproofx_set_mark:Nnn {
115   \dim_set:Nn \l__ebproofx_transit_dim { #3 }
116   \prop_put:cnV { \__ebproofx_marks:N #1 } { #2 }

```

```

117   \l__ebproofx_transit_dim
118 }

```

(End of definition for __ebproofx_set_mark:Nnn.)

__ebproofx_mark:Nn Getting the value of a mark simply consists in getting an item in a property list.

```

119 \cs_new:Nn \__ebproofx_mark:Nn {
120   \prop_item:cn { \__ebproofx_marks:N #1 } { #2 }
121 }

```

(End of definition for __ebproofx_mark:Nn.)

__ebproofx_shift_x:Nn This function shifts the marks by a specified amount, without modifying the box.

```

122 \cs_new:Nn \__ebproofx_shift_x:Nn {
123   \prop_map_inline:cn { \__ebproofx_marks:N #1 } {
124     \__ebproofx_set_mark:Nnn #1 { ##1 } { ##2 + #2 }
125   }
126 }

```

(End of definition for __ebproofx_shift_x:Nn.)

__ebproofx_enlarge_conclusion:NN This function moves the left and right marks of the first tree so that they are at least as far from the axis as they are in the second tree. For instance we get the following:

L — A ————— R	box 1 before
L ————— A — R	box 2 before
L ————— A ————— R	box 1 after

The contents of the trees are unchanged.

```

127 \cs_new:Nn \__ebproofx_enlarge_conclusion:NN {
128   \dim_set:Nn \l_tmpa_dim { \__ebproofx_mark:Nn #1 {axis}
129     + \__ebproofx_mark:Nn #2 {left} - \__ebproofx_mark:Nn #2 {axis} }
130   \dim_compare:nNt { \l_tmpa_dim } < { \__ebproofx_mark:Nn #1 {left} } {
131     \__ebproofx_set_mark:Nnn #1 {left} { \l_tmpa_dim } }
132   \dim_set:Nn \l_tmpa_dim { \__ebproofx_mark:Nn #1 {axis}
133     + \__ebproofx_mark:Nn #2 {right} - \__ebproofx_mark:Nn #2 {axis} }
134   \dim_compare:nNt { \l_tmpa_dim } > { \__ebproofx_mark:Nn #1 {right} } {
135     \__ebproofx_set_mark:Nnn #1 {right} { \l_tmpa_dim } }
136 }

```

(End of definition for __ebproofx_enlarge_conclusion:NN.)

A.2.2 Building blocks

__ebproofx_make_simple:Nn Make a tree with explicit material in horizontal mode. Set the left and right marks to extremal positions and set the axis in the middle.

```

137 \cs_new:Nn \__ebproofx_make_simple:Nn {
138   \hbox_set:cn { \__ebproofx_box:N #1 } { #2 }
139   \__ebproofx_set_mark:Nnn #1 { left } { 0pt }
140   \__ebproofx_set_mark:Nnn #1 { axis } { \box_wd:c { \__ebproofx_box:N #1 } / 2 }
141   \__ebproofx_set_mark:Nnn #1 { right } { \box_wd:c { \__ebproofx_box:N #1 } }
142 }

```

(End of definition for __ebproofx_make_simple:Nn.)

`\__ebproofx_make_split:Nnn` Make a tree with explicit material in horizontal mode, split in two parts. Set the left and right marks to extremal positions and set the axis between the two parts.

```

143 \cs_new:Nn \__ebproofx_make_split:Nnn {
144   \__ebproofx_set_mark:Nnn #1 { left } { 0pt }
145   \hbox_set:cn { \__ebproofx_box:N #1 } { #2 }
146   \__ebproofx_set_mark:Nnn #1 { axis } { \box_wd:c { \__ebproofx_box:N #1 } }
147   \hbox_set:cn { \__ebproofx_box:N #1 } { \hbox_unpack:c { \__ebproofx_box:N #1 } #3 }
148   \__ebproofx_set_mark:Nnn #1 { right } { \box_wd:c { \__ebproofx_box:N #1 } }
149 }

```

(End of definition for __ebproofx_make_split:Nnn.)

`\__ebproofx_make_vertical:Nnnn` Make a tree with explicit material in vertical mode, using an explicit width and axis.

```

150 \cs_new:Nn \__ebproofx_make_vertical:Nnnn {
151   \__ebproofx_set_mark:Nnn #1 { left } { 0pt }
152   \__ebproofx_set_mark:Nnn #1 { axis } { #2 }
153   \__ebproofx_set_mark:Nnn #1 { right } { #3 }
154   \vbox_set:cn { \__ebproofx_box:N #1 } {
155     \dim_set:Nn \tex_hsize:D { \__ebproofx_mark:Nn #1 {right} }
156     #4
157   }
158   \box_set_wd:cn { \__ebproofx_box:N #1 } { \__ebproofx_mark:Nn #1 {right} }
159 }

```

(End of definition for __ebproofx_make_vertical:Nnnn.)

A.2.3 Assembling boxes

`\__ebproofx_extend:Nnnnn` Extend a tree box. The marks are shifted so that alignment is preserved. The arguments are dimensions for the left, top, right and bottom sides respectively.

```

160 \cs_new:Nn \__ebproofx_extend:Nnnnn {
161   \dim_compare:nNnF { #2 } = { 0pt } {
162     \hbox_set:cn { \__ebproofx_box:N #1 } {
163       \skip_horizontal:n { #2 }
164       \box_use:c { \__ebproofx_box:N #1 }
165     }
166     \__ebproofx_shift_x:Nn #1 { #2 }
167   }
168   \box_set_ht:Nn #1 { \box_ht:c { \__ebproofx_box:N #1 } + #3 }
169   \box_set_wd:Nn #1 { \box_wd:c { \__ebproofx_box:N #1 } + #4 }
170   \box_set_dp:Nn #1 { \box_dp:c { \__ebproofx_box:N #1 } + #5 }
171 }

```

(End of definition for __ebproofx_extend:Nnnnn.)

`\__ebproofx_append_right:NnN` Append the contents of the second tree to the first one on the right, with matching baselines. The marks of both trees are preserved. The middle argument specifies the space to insert between boxes.

```

172 \cs_new:Nn \__ebproofx_append_right:NnN {
173   \hbox_set:cn { \__ebproofx_box:N #1 } {
174     \box_use:c { \__ebproofx_box:N #1 }
175     \dim_compare:nNnF { #2 } = { 0pt } { \skip_horizontal:n { #2 } }
176     \box_use:c { \__ebproofx_box:N #3 }
177   }
178 }

```

(End of definition for __ebproofx_append_right:NnN.)

__ebproofx_append_left:NnN Append the contents of the second tree to the first one on the left, with matching baselines. The marks of the first tree are shifted accordingly. The middle argument specifies the space to insert between boxes.

```

179 \cs_new:Nn \__ebproofx_append_left:NnN {
180   \__ebproofx_shift_x:Nn #1 { \box_wd:c { \__ebproofx_box:N #3 } + #2 }
181   \hbox_set:cn { \__ebproofx_box:N #1 } {
182     \box_use:c { \__ebproofx_box:N #3 }
183     \dim_compare:nNnF { #2 } = { 0pt } { \skip_horizontal:n { #2 } }
184     \box_use:c { \__ebproofx_box:N #1 }
185   }
186 }
```

(End of definition for __ebproofx_append_left:NnN.)

__ebproofx_align:NnN Shift one of two trees to the right so that their axes match. The marks of the one that is shifted are updated accordingly.

```

187 \cs_new:Nn \__ebproofx_align:NnN {
188   \dim_set:Nn \l_tmpa_dim
189   { \__ebproofx_mark:Nn #2 {axis} - \__ebproofx_mark:Nn #1 {axis} }
190   \dim_compare:nNnTF \l_tmpa_dim < { 0pt } {
191     \__ebproofx_extend:Nnnnn #2 { -\l_tmpa_dim } { 0pt } { 0pt } { 0pt }
192   } {
193     \__ebproofx_extend:Nnnnn #1 { \l_tmpa_dim } { 0pt } { 0pt } { 0pt }
194   }
195 }
```

(End of definition for __ebproofx_align:NnN.)

__ebproofx_append_above:NnN Append the contents of the second tree above the first one, with matching axes. The marks of the first tree are preserved.

```

196 \cs_new:Nn \__ebproofx_append_above:NnN {
197   \__ebproofx_align:NnN #1 #2
198   \vbox_set:cn { \__ebproofx_box:N #1 } {
199     \box_use:c { \__ebproofx_box:N #2 }
200     \tex_prevdepth:D -1000pt
201     \box_use:c { \__ebproofx_box:N #1 }
202   }
203 }
```

(End of definition for __ebproofx_append_above:NnN.)

__ebproofx_append_below:NnN Append the contents of the second tree below the first one, with matching axes. The marks of the first tree are preserved.

```

204 \cs_new:Nn \__ebproofx_append_below:NnN {
205   \__ebproofx_align:NnN #1 #2
206   \vbox_set_top:cn { \__ebproofx_box:N #1 } {
207     \box_use:c { \__ebproofx_box:N #1 }
208     \tex_prevdepth:D -1000pt
209     \box_use:c { \__ebproofx_box:N #2 }
210   }
211 }
```

(End of definition for __ebproofx_append_below:NnN.)

`\__ebproofx_overlay:NN` Append the second tree as an overlay over the first one, so that the baselines and axes match. The bounding box of the result adjusts to contain both trees.

```

212 \cs_new:Nn \__ebproofx_overlay:NN {
213   \__ebproofx_align:NN #1 #2
214   \hbox_set:cn { \__ebproofx_box:N #1 } {
215     \hbox_overlap_right:n { \box_use:c { \__ebproofx_box:N #1 } }
216     \box_use:c { \__ebproofx_box:N #2 }
217     \dim_compare:nNnT
218       { \box_wd:c { \__ebproofx_box:N #2 } } < { \box_wd:c { \__ebproofx_box:N #1 } }
219       { \skip_horizontal:n
220         { \box_wd:c { \__ebproofx_box:N #1 } - \box_wd:c { \__ebproofx_box:N #2 } } }
221   }
222 }
```

(End of definition for `\__ebproofx_overlay:NN`.)

`\__ebproofx_vcenter:N` Shift the material in a tree vertically so that the height and depth are equal (like $\TeX$'s `\vcenter` but around the baseline).

```

223 \cs_new:Nn \__ebproofx_vcenter:N {
224   \dim_set:Nn \l_tmpa_dim
225     { ( \box_ht:c { \__ebproofx_box:N #1 } - \box_dp:c { \__ebproofx_box:N #1 } ) / 2 }
226   \box_set_eq:Nc \l_tmpa_box { \__ebproofx_box:N #1 }
227   \hbox_set:cn { \__ebproofx_box:N #1 }
228     { \box_move_down:nn { \l_tmpa_dim } { \box_use:N \l_tmpa_box } }
229 }
```

(End of definition for `\__ebproofx_vcenter:N`.)

A.3 Making inferences

The following commands use the parameters defined at the beginning of the package for actually building proof trees using the commands defined above.

`\_ebproofx_append_vertical:NN` Append the contents of the second tree above or below the first one, depending on current settings. Axes are aligned and the marks of the first tree are preserved.

```

230 \cs_new:Nn \_ebproofx_append_vertical:NN {
231   \bool_if:NTF \l__ebproofx_updown_bool
232     { \_ebproofx_append_below:NN #1 #2 }
233     { \_ebproofx_append_above:NN #1 #2 }
234 }
```

(End of definition for `\_ebproofx_append_vertical:NN`.)

`\__ebproofx_make_rule_for:NNN` Make a box containing an inference rule with labels, using the current settings. The width and axis position are taken as those of the conclusion of another tree box. The third argument is used as a temporary register for building labels.

```

235 \cs_new:Nn \__ebproofx_make_rule_for:NNN {
```

Build the rule.

```

236   \__ebproofx_make_vertical:Nnnn #1
237   { \__ebproofx_mark:Nn #2 {axis} - \__ebproofx_mark:Nn #2 {left} }
238   { \__ebproofx_mark:Nn #2 {right} - \__ebproofx_mark:Nn #2 {left} }
239   {
240     \skip_vertical:N \l__ebproofx_rule_margin_dim
```

```

241     \tl_if_empty:NF { \l__ebproofx_rule_code_tl } {
242       \tl_use:N \l__ebproofx_rule_code_tl
243       \skip_vertical:N \l__ebproofx_rule_margin_dim
244     }
245   }
246   \__ebproofx_vcenter:N #1

```

Append the left label.

```

247   \tl_if_blank:VF \l__ebproofx_left_label_tl {
248     \__ebproofx_make_simple:Nn #3 {
249       \box_move_down:nn { \l__ebproofx_label_axis_dim } { \hbox:n {
250         \cs_set_eq:NN \inserttext \l__ebproofx_left_label_tl
251         \tl_use:N \l__ebproofx_left_label_template_tl
252       } }
253     }
254     \box_set_ht:cn { \__ebproofx_box:N #3 } { 0pt }
255     \box_set_dp:cn { \__ebproofx_box:N #3 } { 0pt }
256     \__ebproofx_append_left:NnN
257     \l__ebproofx_c_box \l__ebproofx_label_separation_dim \l__ebproofx_d_box
258   }

```

Append the right label.

```

259   \tl_if_blank:VF \l__ebproofx_right_label_tl {
260     \__ebproofx_make_simple:Nn #3 {
261       \box_move_down:nn { \l__ebproofx_label_axis_dim } { \hbox:n {
262         \cs_set_eq:NN \inserttext \l__ebproofx_right_label_tl
263         \tl_use:N \l__ebproofx_right_label_template_tl
264       } }
265     }
266     \box_set_ht:cn { \__ebproofx_box:N #3 } { 0pt }
267     \box_set_dp:cn { \__ebproofx_box:N #3 } { 0pt }
268     \__ebproofx_append_right:NnN
269     \l__ebproofx_c_box \l__ebproofx_label_separation_dim \l__ebproofx_d_box
270   }
271 }

```

(End of definition for __ebproofx_make_rule_for:NNN.)

A.4 Stack-based interface

A.4.1 The stack

Logically, box structures are stored on a stack. However, $\text{\TeX}$ does not provide data structures for that and the grouping mechanism is not flexible enough, so we encode them using what we actually have. A stack for boxes is implemented using a global `\g__ebproofx_stack_box` that contains all the boxes successively. A sequence `\g__ebproofx_stack_seq` is used to store the dimensions property lists textually. We maintain a counter `\g__ebproofx_level_int` with the number of elements on the stack, for consistency checks.

```

272 \int_new:N \g__ebproofx_level_int
273 \box_new:N \g__ebproofx_stack_box
274 \seq_new:N \g__ebproofx_stack_seq

```

`\__ebproofx_clear_stack:` Clear the stack.

```

275 \cs_new:Nn \__ebproofx_clear_stack: {

```

```

276 \int_gset:Nn \g__ebproofx_level_int { 0 }
277 \hbox_gset:Nn \g__ebproofx_stack_box { }
278 \seq_gclear:N \g__ebproofx_stack_seq
279 }

```

(End of definition for __ebproofx_clear_stack:.)

__ebproofx_push:N Push the contents of a register on the stack.

```

280 \cs_new:Nn \__ebproofx_push:N {
281   \int_gincr:N \g__ebproofx_level_int
282   \hbox_gset:Nn \g__ebproofx_stack_box
283   { \hbox_unpack:N \g__ebproofx_stack_box \box_use:c { \__ebproofx_box:N #1 } }
284   \seq_gput_left:Nv \g__ebproofx_stack_seq
285   { \__ebproofx_marks:N #1 }
286 }

```

(End of definition for __ebproofx_push:N.)

__ebproofx_pop:N Pop the value from the top of the stack into a register.

```

287 \cs_new:Nn \__ebproofx_pop:N {
288   \int_compare:nNnTF { \g__ebproofx_level_int } > { 0 } {
289     \int_gdecr:N \g__ebproofx_level_int
290     \hbox_gset:Nn \g__ebproofx_stack_box {
291       \hbox_unpack:N \g__ebproofx_stack_box
292       \box_gset_to_last:N \g_tmpa_box
293     }
294     \box_set_eq_drop:cN { \__ebproofx_box:N #1 } \g_tmpa_box
295     \seq_gpop_left:NN \g__ebproofx_stack_seq \l_tmpa_tl
296     \tl_set_eq:cN { \__ebproofx_marks:N #1 } \l_tmpa_tl
297   } {
298     \PackageError{ebproofx}{Missing~premiss~in~a~proof~tree}{}
299     \__ebproofx_clear:N #1
300   }
301 }

```

(End of definition for __ebproofx_pop:N.)

A.4.2 Assembling trees

```

302 \__ebproofx_new:N \l__ebproofx_a_box
303 \__ebproofx_new:N \l__ebproofx_b_box
304 \__ebproofx_new:N \l__ebproofx_c_box
305 \__ebproofx_new:N \l__ebproofx_d_box

```

__ebproofx_join_horizontal:n Join horizontally a number of elements at the top of the stack. If several trees are joined, use the left mark of the left tree, the right mark of the right tree and set the axis in the middle of these marks.

```

306 \cs_new:Nn \__ebproofx_join_horizontal:n {
307   \int_case:nnF { #1 } {
308     { 0 } {
309       \group_begin:
310       \__ebproofx_clear:N \l__ebproofx_a_box
311       \__ebproofx_push:N \l__ebproofx_a_box
312       \group_end:
313     }

```

```

314 { 1 } { }
315 } {
316   \group_begin:
317   \__ebproofx_pop:N \l__ebproofx_a_box
318   \prg_replicate:nn { #1 - 1 } {
319     \__ebproofx_pop:N \l__ebproofx_b_box
320     \__ebproofx_append_left:NnN
321       \l__ebproofx_a_box \l__ebproofx_separation_dim \l__ebproofx_b_box
322   }
323   \__ebproofx_set_mark:Nnn \l__ebproofx_a_box { left }
324   { \__ebproofx_mark:Nn \l__ebproofx_b_box { left } }
325   \__ebproofx_set_mark:Nnn \l__ebproofx_a_box { axis }
326   { ( \__ebproofx_mark:Nn \l__ebproofx_a_box { left }
327     + \__ebproofx_mark:Nn \l__ebproofx_a_box { right } ) / 2 }
328   \__ebproofx_push:N \l__ebproofx_a_box
329   \group_end:
330 }
331 }

```

(End of definition for __ebproofx_join_horizontal:n.)

__ebproofx_join_vertical: Join vertically the two elements at the top of the stack, with a horizontal rule of the appropriate size.

```

332 \cs_new:Nn \__ebproofx_join_vertical: {
333   \group_begin:
334   \__ebproofx_pop:N \l__ebproofx_a_box
335   \__ebproofx_pop:N \l__ebproofx_b_box
336   \__ebproofx_enlarge_conclusion:NN \l__ebproofx_b_box \l__ebproofx_a_box
337   \__ebproofx_make_rule_for:NNN \l__ebproofx_c_box \l__ebproofx_b_box
338     \l__ebproofx_d_box
339   \__ebproofx_append_vertical:NN \l__ebproofx_a_box \l__ebproofx_c_box
340   \__ebproofx_append_vertical:NN \l__ebproofx_a_box \l__ebproofx_b_box
341   \__ebproofx_push:N \l__ebproofx_a_box
342   \group_end:
343 }

```

(End of definition for __ebproofx_join_vertical:.)

A.4.3 High-level commands

__ebproofx_statement_parse:w An auxiliary function for parsing the argument in __ebproofx_push_statement:n.

```

344 \cs_new:Npn \__ebproofx_statement_parse:w #1 & #2 & #3 \q_stop {
345   \tl_if_empty:nTF { #3 } {
346     \__ebproofx_make_simple:Nn \l__ebproofx_a_box
347     { \cs_set:Npn \inserttext { #1 } \tl_use:N \l__ebproofx_template_tl }
348   } {
349     \__ebproofx_make_split:Nnn \l__ebproofx_a_box
350     { \cs_set:Npn \inserttext { #1 } \tl_use:N \l__ebproofx_left_template_tl }
351     { \cs_set:Npn \inserttext { #2 } \tl_use:N \l__ebproofx_right_template_tl }
352   }
353   \__ebproofx_push:N \l__ebproofx_a_box
354 }

```

(End of definition for __ebproofx_statement_parse:w.)

`\__ebproofx_push_statement:n` Push a box with default formatting, using explicit alignment if the code contains a & character

```

355 \cs_new:Nn \__ebproofx_push_statement:n {
356   \__ebproofx_statement_parse:w #1 & & \q_stop
357 }

```

(End of definition for `\__ebproofx_push_statement:n`.)

A.5 Document interface

A.5.1 Functions to define statements

The `\g__ebproofx_statements_seq` variable contains the list of all defined statements. For each statement X , there is a document command `\ebproofxX` and the alias X is defined when entering a `prooftree` environment.

```

358 \seq_new:N \g__ebproofx_statements_seq

```

`\__ebproofx_setup_statements:` Install the aliases for statements, saving the original value of the control sequences.

```

359 \cs_new:Nn \__ebproofx_setup_statements: {
360   \seq_map_inline:Nn \g__ebproofx_statements_seq {
361     \cs_set_eq:cc { ebproofx_saved_ ##1 } { ##1 }
362     \cs_set_eq:cc { ##1 } { ebproofx ##1 }
363   }
364 }

```

(End of definition for `\__ebproofx_setup_statements:`.)

`\__ebproofx_restore_statements:` Restore the saved meanings of the control sequences. This is useful when interpreting user-provided code in statement arguments. The meanings are automatically restored when leaving a `prooftree` environment because of grouping.

```

365 \cs_new:Nn \__ebproofx_restore_statements: {
366   \seq_map_inline:Nn \g__ebproofx_statements_seq {
367     \cs_set_eq:cc { ##1 } { ebproofx_saved_ ##1 }
368   }
369 }

```

(End of definition for `\__ebproofx_restore_statements:`.)

`\__ebproofx_new_statement:nnn` Define a new statement. The first argument is the name, the second one is an argument specifier as used by `xparse` and the third one is the body of the command.

```

370 \cs_new:Nn \__ebproofx_new_statement:nnn {
371   \exp_args:Nc \NewDocumentCommand { ebproofx#1 }{ #2 } { #3 }
372   \seq_gput_right:Nn \g__ebproofx_statements_seq { #1 }
373 }

```

(End of definition for `\__ebproofx_new_statement:nnn`.)

`\__ebproofx_new_deprecated_statement:nnnn` Define a deprecated statement. The syntax is the same as above except that an extra argument in third position indicates what should be used instead. The effect is the same except that a warning message is issued the first time the statement is used.

```

374 \cs_new:Nn \__ebproofx_new_deprecated_statement:nnnn {
375   \cs_new:cpn { ebproofx_#1_warning: } {
376     \PackageWarning { ebproofx } { \token_to_str:c{#1}~is~deprecated,~#3 }
377     \cs_gset:cn { ebproofx_#1_warning: } { }
378   }

```

```

379 \__ebproofx_new_statement:nnn { #1 } { #2 }
380 { \use:c { ebproofx_#1_warning: } #4 }
381 }

```

(End of definition for __ebproofx_new_deprecated_statement:nnnn.)

A.5.2 Basic commands

`\ebproofxset` This is a simple wrapper around `\keys_set:nn`.

```

\set
382 \__ebproofx_new_statement:nnn { set } { m } {
383 \keys_set:nn { ebproofx } { #1 }
384 }

```

(End of definition for `\ebproofxset` and `\set`. These functions are documented on page ??.)

`\hypo` This is mostly a wrapper around `\ebproofx_push_statement:n`, with material to handle options and the statements macros.

```

385 \__ebproofx_new_statement:nnn { hypo } { O{ m } {
386 \group_begin:
387 \__ebproofx_restore_statements:
388 \keys_set:nn { ebproofx } { #1 }
389 \__ebproofx_push_statement:n { #2 }
390 \group_end:
391 }

```

(End of definition for `\hypo`. This function is documented on page ??.)

`\infer` This is a bit more involved than `\hypo` because we have to handle rule style options and joining.

```

392 \__ebproofx_new_statement:nnn { infer } { O{ m O{ m } {
393 \group_begin:
394 \__ebproofx_restore_statements:
395 \keys_set:known:nnN { ebproofx / rule~style } { #1 } \l_tmpa_tl
396 \keys_set:nV { ebproofx } \l_tmpa_tl
397 \tl_set:Nn \l__ebproofx_right_label_tl { #3 }
398 \__ebproofx_join_horizontal:n { #2 }
399 \__ebproofx_push_statement:n { #4 }
400 \__ebproofx_join_vertical:
401 \group_end:
402 }

```

(End of definition for `\infer`. This function is documented on page 2.)

`\ellipsis` An ellipsis is made by hand using vertical leaders to render the dots after rendering the label.

```

403 \__ebproofx_new_statement:nnn { ellipsis } { m m } {
404 \group_begin:
405 \__ebproofx_restore_statements:
406 \tl_clear:N \l__ebproofx_rule_code_tl
407 \__ebproofx_make_split:Nnn \l__ebproofx_a_box { } {
408 \vbox_set:Nn \l_tmpa_box {
409 \skip_vertical:n { 1.2ex }
410 \hbox:n { \tex_ignorespaces:D #1 }
411 \skip_vertical:n { 1.2ex }
412 }
413 \vbox_to_ht:nn { \box_ht:N \l_tmpa_box } {
414 \tex_xleaders:D \vbox_to_ht:nn { .8ex }

```

```

415         { \tex_vss:D \hbox:n { . } \tex_vss:D }
416         \tex_vfill:D
417     }
418     \hbox_overlap_right:n { ~ \box_use:N \l_tmpa_box }
419 }
420 \__ebproofx_push:N \l__ebproofx_a_box
421 \__ebproofx_join_vertical:
422 \__ebproofx_push_statement:n {#2}
423 \__ebproofx_join_vertical:
424 \group_end:
425 }

```

(End of definition for \ellipsis. This function is documented on page ??.)

A.5.3 Modifying trees

\rewrite Rewrite the box at the top of the stack while preserving its dimensions and marks. The code is typeset in horizontal mode, with control sequences to access the original box and its marks.

```

426 \__ebproofx_new_statement:nnn { rewrite } { m } {
427   \group_begin:
428   \__ebproofx_restore_statements:
429   \__ebproofx_pop:N \l__ebproofx_a_box
430   \box_set_eq:Nc \l_tmpa_box { \__ebproofx_box:N \l__ebproofx_a_box }
431   \hbox_set:Nn \l_tmpb_box {
432     \cs_set_eq:NN \treebox \l_tmpa_box
433     \cs_set:Npn \treemark { \__ebproofx_mark:Nn \l__ebproofx_a_box }
434     { #1 }
435   }
436   \box_set_wd:Nn \l_tmpb_box { \box_wd:c { \__ebproofx_box:N \l__ebproofx_a_box } }
437   \box_set_ht:Nn \l_tmpb_box { \box_ht:c { \__ebproofx_box:N \l__ebproofx_a_box } }
438   \box_set_dp:Nn \l_tmpb_box { \box_dp:c { \__ebproofx_box:N \l__ebproofx_a_box } }
439   \box_set_eq:cN { \__ebproofx_box:N \l__ebproofx_a_box } \l_tmpb_box
440   \__ebproofx_push:N \l__ebproofx_a_box
441   \group_end:
442 }

```

(End of definition for \rewrite. This function is documented on page ??.)

\delims Insert \left and \right delimiters without changing the alignment.

```

443 \__ebproofx_new_statement:nnn { delims } { m m } {
444   \group_begin:
445   \__ebproofx_restore_statements:
446   \__ebproofx_pop:N \l__ebproofx_a_box
447   \hbox_set:Nn \l_tmpa_box
448     { $ \tex_vcenter:D { \box_use:c { \__ebproofx_box:N \l__ebproofx_a_box } } $ }
449   \dim_set:Nn \l_tmpa_dim
450     { \box_ht:N \l_tmpa_box - \box_ht:c { \__ebproofx_box:N \l__ebproofx_a_box } }
451   \hbox_set:cn { \__ebproofx_box:N \l__ebproofx_a_box } {
452     $ #1 \tex_vrule:D
453       height \box_ht:N \l_tmpa_box depth \box_dp:N \l_tmpa_box width 0pt
454     \tex_right:D . $
455   }
456   \__ebproofx_shift_x:Nn \l__ebproofx_a_box
457     { \box_wd:c { \__ebproofx_box:N \l__ebproofx_a_box } }

```

```

458 \hbox_set:cn { \__ebproofx_box:N \l__ebproofx_a_box } {
459   \hbox_unpack:c { \__ebproofx_box:N \l__ebproofx_a_box }
460   $ \tex_left:D . \box_use:N \l_tmpa_box #2 $
461 }
462 \hbox_set:cn { \__ebproofx_box:N \l__ebproofx_a_box }
463   { \box_move_down:nn { \l_tmpa_dim }
464     { \box_use:c { \__ebproofx_box:N \l__ebproofx_a_box } } }
465 \__ebproofx_push:N \l__ebproofx_a_box
466 \group_end:
467 }

```

(End of definition for \delims. This function is documented on page ??.)

\overlay Pop two trees and append the second tree as an overlay over the first one, so that the baselines and axes match. The bounding box of the result adjusts to contain both trees.

```

468 \__ebproofx_new_statement:nnn { overlay } { } {
469   \group_begin:
470   \__ebproofx_pop:N \l__ebproofx_a_box
471   \__ebproofx_pop:N \l__ebproofx_b_box
472   \__ebproofx_overlay:NN \l__ebproofx_a_box \l__ebproofx_b_box
473   \__ebproofx_push:N \l__ebproofx_a_box
474   \group_end:
475 }

```

(End of definition for \overlay. This function is documented on page ??.)

A.5.4 The extension

```

476 \int_new:N \g__ebproofx_sublevel_int
477 \box_new:N \g__ebproofx_substack_box
478 \seq_new:N \g__ebproofx_substack_seq
479
480 \cs_new:Nn \__ebproofx_clear_substack:
481 {
482   \int_gset:Nn \g__ebproofx_sublevel_int { 0 }
483   \hbox_gset:Nn \g__ebproofx_substack_box { }
484   \seq_gclear:N \g__ebproofx_substack_seq
485 }
486
487 \cs_new:Nn \__ebproofx_subpush:N
488 {
489   \int_gincr:N \g__ebproofx_sublevel_int
490   \hbox_gset:Nn \g__ebproofx_substack_box
491     { \hbox_unpack:N \g__ebproofx_substack_box \box_use:c { \__ebproofx_box:N #1 } }
492   \seq_gput_left:Nv \g__ebproofx_substack_seq
493     { \__ebproofx_marks:N #1 }
494 }
495
496 \cs_new:Nn \__ebproofx_subpop:N
497 {
498   \int_compare:nNnTF { \g__ebproofx_sublevel_int } > { 0 }
499   {
500     \int_gdecr:N \g__ebproofx_sublevel_int
501     \hbox_gset:Nn \g__ebproofx_substack_box {
502       \hbox_unpack:N \g__ebproofx_substack_box

```



```

503     \box_gset_to_last:N \g_tmpa_box
504   }
505   \box_set_eq_drop:cN { \__ebproofx_box:N #1 } \g_tmpa_box
506   \seq_gpop_left:NN \g__ebproofx_substack_seq \l_tmpa_tl
507   \tl_set_eq:cN { \__ebproofx_marks:N #1 } \l_tmpa_tl
508 }
509 { \PackageError{ebproofx}{Missing~premiss~in~a~proof~tree}{ } \__ebproofx_clear:N #1 }
510 }
511
512 \cs_new:Nn \__ebproofx_append_subvertical:NN
513 {
514   \bool_if:NTF \l__ebproofx_updown_bool
515   { \__ebproofx_append_above:NN #1 #2 }
516   { \__ebproofx_append_below:NN #1 #2 }
517 }
518
519 \cs_new:Nn \__ebproofx_join_subvertical:n
520 {
521   \group_begin:
522   \__ebproofx_subpop:N \l__ebproofx_a_box
523   \prg_replicate:nn { #1 - 1 }
524   {
525     \__ebproofx_subpop:N \l__ebproofx_b_box
526     \__ebproofx_enlarge_conclusion:NN \l__ebproofx_b_box \l__ebproofx_a_box
527     \__ebproofx_enlarge_conclusion:NN \l__ebproofx_a_box \l__ebproofx_b_box
528
529     \__ebproofx_make_vertical:Nnnn \l__ebproofx_c_box
530     {
531       \__ebproofx_mark:Nn \l__ebproofx_b_box {axis}
532       - \__ebproofx_mark:Nn \l__ebproofx_b_box {left}
533     }
534     {
535       \__ebproofx_mark:Nn \l__ebproofx_b_box {right}
536       - \__ebproofx_mark:Nn \l__ebproofx_b_box {left}
537     }
538     { \skip_vertical:N \l__ebproofx_rule_margin_dim }
539     \__ebproofx_vcenter:N \l__ebproofx_b_box
540     \__ebproofx_append_subvertical:NN \l__ebproofx_a_box \l__ebproofx_c_box
541
542     \__ebproofx_append_subvertical:NN \l__ebproofx_a_box \l__ebproofx_b_box
543   }
544   \__ebproofx_push:N \l__ebproofx_a_box
545   \group_end:
546 }
547
548 \cs_new:Nn \__ebproofx_renew_statement:nnn
549 {
550   \exp_args:Nc \RenewDocumentCommand { ebproofx#1 }{ #2 } { #3 }
551   \seq_gput_right:Nn \g__ebproofx_statements_seq { #1 }
552 }
553
554 \cs_generate_variant:Nn \clist_map_inline:nn { xn }
555 \__ebproofx_renew_statement:nnn { infer } { 0{ } m 0{ } m }
556 {

```

```

557 \group_begin:
558 \__ebproofx_restore_statements:
559 \keys_set_known:nnN { ebproofx / rule~style } { #1 } \l_tmpa_tl
560 \keys_set:nV { ebproofx } \l_tmpa_tl
561 \tl_set:Nn \l__ebproofx_right_label_tl { #3 }
562
563 \__ebproofx_clear_substack:
564 \clist_map_inline:xn { \clist_reverse:n { #2 } }
565 {
566   \__ebproofx_join_horizontal:n { ##1 }
567   \__ebproofx_pop:N \l__ebproofx_a_box
568   \__ebproofx_subpush:N \l__ebproofx_a_box
569 }
570 \__ebproofx_join_subvertical:n { \clist_count:n { #2 } }
571
572 \__ebproofx_push_statement:n { #4 }
573 \__ebproofx_join_vertical:
574 \group_end:
575 }
576 % \end{macrocode}
577 %
578 %
579 % \subsubsection{Deprecated statements}
580 %
581 % These statements were defined in versions 1.x of the package, they are
582 % preserved for temporary upwards compatibility and will be removed in a
583 % future version.
584 % \begin{macrocode}
585 \__ebproofx_new_deprecated_statement:nnnn { Alter } { m }
586 { use~\token_to_str:c{rewrite}~instead } { \ebproofxrewrite{ #1 \box\treebox } }
587 \__ebproofx_new_deprecated_statement:nnnn { Delims } { }
588 { use~\token_to_str:c{delims}~instead } { \ebproofxdelims }
589 \__ebproofx_new_deprecated_statement:nnnn { Ellipsis } { }
590 { use~\token_to_str:c{ellipsis}~instead } { \ebproofxellipsis }
591 \__ebproofx_new_deprecated_statement:nnnn { Hypo } { }
592 { use~\token_to_str:c{hypo}~instead } { \ebproofxhypo }
593 \__ebproofx_new_deprecated_statement:nnnn { Infer } { }
594 { use~\token_to_str:c{infer}~instead } { \ebproofxinfer }

```

A.5.5 Environment interface

The stack is initialised globally. The prooftree environment does not clear the stack, instead it saves the initial level in order to check that statements are properly balanced. This allows for nested uses of the environment, if it ever happens to be useful.

```

595 \__ebproofx_clear_stack:
596 \tl_new:N \l__ebproofx_start_level_tl

```

prooftree The prooftree environment.

```

prooftree* 597 \NewDocumentEnvironment { prooftree } { s O{ } } {
598   \group_align_safe_begin:
599   \keys_set_known:nnN { ebproofx / proof~style } { #2 } \l_tmpa_tl
600   \keys_set:nV { ebproofx } \l_tmpa_tl
601   \tl_set:Nx \l__ebproofx_start_level_tl { \int_use:N \g__ebproofx_level_int }
602   \vbox_set:Nw \l_tmpa_box

```

```

603   \__ebproofx_setup_statements:
604 } {
605   \vbox_set_end:
606   \__ebproofx_pop:N \l__ebproofx_a_box
607   \int_compare:nNnF { \g__ebproofx_level_int } = { \tl_use:N \l__ebproofx_start_level_tl } {
608     \PackageError{ebproofx}{Malformed~proof~tree}{
609       Some~hypotheses~were~declared~but~not~used~in~this~tree.}
610   }
611   \IfBooleanTF { #1 } {
612     \[ \box_use:c { \__ebproofx_box:N \l__ebproofx_a_box } \]
613     \ignorespacesafterend
614   } {
615     \hbox_unpack:N \c_empty_box
616     \bool_if:NTF \l__ebproofx_center_bool {
617       \hbox:n { $ \tex_vcenter:D { \box_use:c { \__ebproofx_box:N \l__ebproofx_a_box } } $ }
618     } {
619       \box_use:c { \__ebproofx_box:N \l__ebproofx_a_box }
620     }
621   }
622   \group_align_safe_end:
623 }

```

A trick for the starred version:

```

624 \cs_new:cpn { prooftree* } { \prooftree* }
625 \cs_new:cpn { endprooftree* } { \endprooftree }

```

(End of definition for prooftree and prooftree. These functions are documented on page ??.)*

InfRule The InfRule environment.

```

626 \NewTblrEnviron{@ruleenv}
627 \SetTblrInner[@ruleenv]{belowsep=0pt,stretch=0}
628 \SetTblrOuter[@ruleenv]{baseline=b}
629 \NewDocumentEnvironment { InfRule } { m O{1} +b }
630 {
631   \begin{@ruleenv}{#2}
632     \smaller{\textsc{#1}} \\\
633     \begin{prooftree} #3 \end{prooftree}
634   \end{@ruleenv}
635 }
636 {}

```

(End of definition for InfRule. This function is documented on page 2.)

```

637 \</package>

```